

GrpHist: Independent Reproduction Report

Validating the published downstream results of arXiv:2603.00143 from the released artifacts

19 August 2026

Contents

1	Executive summary	3
2	What the paper claims	4
2.1	The thesis	4
2.2	The method	4
2.3	The headline results	4
3	Technology stack	5
3.1	The paper’s stack	5
3.2	Our stack	5
4	Artifacts: what is actually published	6
4.1	The released region-level graphs are not the ones in Table 2	6
4.2	Integrity of what we did download	6
5	Results	8
5.1	Cell-level tasks — exact paper targets available	8
5.2	Region-level tasks — Figure 4b	8
5.3	Structural claims verified	9
5.4	Coverage	9
6	Measured runtimes	10
7	What we did differently, and why	11
7.1	Reconstructed NuCLS folds — the one deviation that moved numbers	11
7.2	Derived the NuCLS super labels	11
7.3	Non-patched rather than patched region graphs	11
7.4	Wrote the PanNuke tissue filter	11
7.5	Ran on CPU with a much newer software stack	12
7.6	Did not pre-train, and did not build the baselines	12
8	Defects found in the released code	13
8.1	Cell labels are misaligned against their embeddings — silent	13
8.2	AUROC assumes every class appears in every fold — silent then fatal	13
8.3	The checkpoint cannot be loaded without a GPU	13
8.4	The normalization transform cannot be pickled	13
8.5	Version incompatibilities	13
8.6	Placeholder paths	14
8.7	Summary of changes made	14

9	A substantive discrepancy: the virtual node	15
10	Other observations	16
11	Conclusions	17
11.1	Recommended next steps	17
12	Appendix: reproduction commands	18

1 Executive summary

This report documents an independent attempt to reproduce the downstream results of **GrapHist: Graph Self-Supervised Learning for Histopathology** (Öğüt et al., arXiv:2603.00143) using only the artifacts the authors released: the pre-trained checkpoint, the published cell-graph datasets, and the code in github.com/ogutsevda/graphist.

The scope was deliberately narrowed to **validating the published numbers from the released weights**. We did not re-run self-supervised pre-training, and we did not re-implement the four comparison baselines (DINOv2, MAE, ACM-bio, ACM-UNI), none of which have training code in the repository. The question this report answers is therefore: *given what the authors published, do their reported downstream results come out again?*

The headline answer is yes, where the data permits. All four PanNuke 40x metrics reproduce to within 0.19 percentage points across both the pan-cancer and breast-only subsets — eight numbers, four of them agreeing to better than a tenth of a point. NuCLS reproduces to within about one point on macro F1 under a reconstructed fold assignment. The two region-level datasets confirm the paper’s patch-size robustness claim. In total **24 of the 89 GrapHist result cells** were produced.

The reproduction was not, however, straightforward. **Four defects in the released code prevent a fresh checkout from producing any cell-level result at all**, and one of them is silent: the released `generate_embs.py` misaligns node labels against node embeddings for every graph except the first in each batch. Any user running the shipped defaults would obtain corrupted cell-level numbers without an error being raised. We also identified a substantive discrepancy between the paper’s described method and its implementation concerning the virtual node, quantified in Section 8.

Two of our earlier assumptions about artifact availability proved wrong on inspection and are corrected here: TCGA-BRCA **is** published as graphs, and BACH is **not**.

2 What the paper claims

2.1 The thesis

GrapHist argues that digital-pathology foundation models are built on the wrong primitive. Vision transformers tile a slide into a regular 14x14-pixel grid, but that grid is not aligned to cells — the entities pathologists actually reason about. The paper’s position is that modelling tissue explicitly as a **cell graph** supplies a biological inductive bias that a grid cannot, and that doing so is more efficient because complexity becomes linear in the number of cells rather than quadratic in the number of tokens.

The claimed payoff is competitive or superior downstream accuracy at roughly a quarter of the parameters, alongside the first large-scale graph benchmark for digital pathology.

2.2 The method

Graph construction. Each 224x224 tile at 20x magnification becomes one graph. Nuclei are detected with StarDist (U-Net backbone, 2D_versatile_he weights), each nucleus becoming a node carrying 96 hand-crafted descriptors spanning colour intensity, morphology, GLCM texture and Fourier shape. Edges come from Delaunay triangulation, pruned above 100 μm on the grounds that longer-range cell-to-cell signalling is implausible, and are weighted by Euclidean distance in microns.

Pre-training. A GraphMAE-style masked autoencoder. A random subset of nodes has its features replaced by a learnable mask token or by another node’s features; an encoder embeds the corrupted graph; the same nodes are re-masked; a single-layer decoder reconstructs the original features. The objective is scaled cosine error with exponent $\gamma = 3$, which down-weights easy reconstructions.

The architecture. Both encoder and decoder use Adaptive Channel Mixing GIN blocks. Each block runs the graph through three channels — low-pass $(I + A_{rw})/2$, high-pass $(I - A_{rw})/2$, and identity — then learns a per-node convex combination of them. This is the paper’s answer to heterophily: tumour microenvironments put dissimilar cells next to each other, so a model that can sharpen at a tumour-stroma boundary while smoothing inside a homogeneous region should beat one that only smooths. Two expressivity additions: a virtual node connected to every other node, and jumping-knowledge concatenation of all layer outputs followed by a linear projection.

Scales. Node embeddings serve cell-level tasks directly; their mean is a patch embedding; attention MIL over patches gives a slide embedding. Three MIL variants are evaluated — ABMIL, additive ABMIL, conjunctive ABMIL — which differ only in where the classifier sits relative to the attention weighting.

2.3 The headline results

Claim	Paper’s evidence
Beats vision SSL on slide subtyping	72.25 vs 66.72 macro F1 on TCGA-BRCA
Beats vision SSL on region subtyping	Up to +9.9 points on BACH, BRACS, BreakHis
Beats vision SSL on cell typing	Wins all six cell-level settings
Beats supervised graph models where labels are scarce	Up to +40 points on slide and region tasks
Far cheaper	9.50M params vs 22.01M / 47.58M; 50 vs 180 / 350 GPU-hours
Patch-size agnostic	Stable from 224px up to whole 4000px RoIs

3 Technology stack

3.1 The paper’s stack

Layer	Component	Version pinned in <code>env.yaml</code>
Language	Python	3.10.0
Deep learning	PyTorch	2.2.2 (CUDA 11.8, cuDNN 8.7)
Graph library	PyTorch Geometric	2.5.2
Segmentation	StarDist (TensorFlow)	unpinned
WSI I/O	OpenSlide	4.0.0
Image processing	scikit-image	0.24.0
Classical ML	scikit-learn	1.5.0
Numerics	NumPy / SciPy	1.26.4 / 1.13.1
Experiment tracking	Weights & Biases	0.17.2
CLI	fire (data scripts), argparse (train/eval)	—
Hardware	NVIDIA A100 80GB and H200 140GB	—

The unpinned StarDist and TensorFlow entries are the most consequential reproducibility gap in the pipeline. StarDist’s version determines which nuclei are detected, and every node, edge and feature downstream inherits that decision. It only matters if graphs are rebuilt from images, but in that case it matters more than any other single choice and is nearly impossible to detect after the fact.

3.2 Our stack

We deliberately did not recreate the pinned environment. The goal was to test whether the released artifacts reproduce the published numbers, and running on a materially different stack is a stronger test of that than running on the authors’ exact versions — if the numbers survive a five-release jump in PyTorch Geometric and a different accelerator, they are not an artifact of the environment.

Layer	Ours	Paper
Python	3.13.13	3.10.0
PyTorch	2.13.0	2.2.2
PyTorch Geometric	2.8.0	2.5.2
scikit-learn	1.9.0	1.5.0
NumPy / SciPy	2.5.2 / 1.18.0	1.26.4 / 1.13.1
Hardware	Apple M3 Pro, 11 cores, 18 GB, CPU only	NVIDIA A100 / H200

The hardware difference is less limiting than it sounds. Embedding generation is a frozen forward pass over small graphs, and the cell-level probes are scikit-learn’s lbfgs solver, which is CPU-bound on any machine. The entire PanNuke pipeline runs in under three minutes on a laptop.

This version gap did surface three incompatibilities, all documented in Section 7.

4 Artifacts: what is actually published

Before any results, an accurate inventory. The paper states that five graph datasets are released. Querying the Hugging Face API for the author’s namespace returns:

Dataset	Graphs	On disk	Status
graph-pannuke	7,208	209 MB	Downloaded, used
graph-nucls	1,694	74 MB	Downloaded, used
graph-bracs	4,493	3.8 GB	Downloaded, used
graph-breakhis	522	30 MB	Downloaded, used
graph-tcga-brca	254 files, no loose .pt	not fetched	Published, likely archived
graph-bach	—	—	Does not exist
graphist.pt + normalization.json	9.50M params	111 MB	Downloaded, used

Two corrections to assumptions we made earlier in this project, both of which change what is reachable:

TCGA-BRCA is published. We had assumed the 11.1-million-graph pre-training corpus was withheld because of its size. It is present as 254 files — almost certainly archives rather than loose graphs. This means the WSI-level column and the survival analysis may be far cheaper to attempt than the ~96 hours of pre-processing we had budgeted, since the graphs would not need rebuilding from raw slides.

BACH is not published. The paper evaluates on BACH throughout Tables 2 and D.9–D.12, but no graph-bach repository exists. Those twelve GrapHist result cells cannot be reached without the original 400 RoI images and a full re-run of pre-processing stages 1–3.

4.1 The released region-level graphs are not the ones in Table 2

The BRACS and BreakHis graph counts — 4,493 and 522 — match the “# Slides” column of the paper’s Table 1, not the “# Patches” column (96,153 and 709). Inspection confirms this: BRACS graphs average 1,888 nodes, up to 15,496, whereas Table 1 quotes 46.96 nodes for a 224px patch. These are whole-RoI graphs.

The repository README acknowledges this directly, warning that the released RoI datasets are built from full images without patching while the main experiments used patched versions. The consequence for reproduction is precise: these graphs reproduce the **Non-Patch** bars of Figure 4b, not the Table 2 / D.9–D.12 columns. Reaching those requires the original images.

4.2 Integrity of what we did download

Every .pt file in PanNuke and NuCLS was deserialized and checked. Results:

Check	PanNuke	NuCLS
Graph count vs Table 1	7,208 (paper 7,215)	1,694 (paper 1,694)
Average nodes	22.64 (paper 22.66)	30.76 (paper 30.76)
Average edges	57.13 (paper 57.16)	78.62 (paper 78.62)
Feature dimension	96	96
NaN or Inf values	0	0
Self-loops, duplicate edges, isolated nodes	0 / 0 / 0	—
Class counts vs Figure A.6	4 of 5 exact	7 of 7 exact
Edge distances within the 100 um cutoff	yes (max 77.8)	yes

BRACS and BreakHis label distributions match Figure A.5 exactly: BreakHis 289 malignant / 233 benign, and all seven BRACS classes (833 / 768 / 752 / 646 / 515 / 505 / 474).

The checkpoint loads with `strict=True` and zero shape mismatches, and was saved at epoch 86 with a best validation loss of 0.2071. `normalization.json` diverges from PanNuke's own statistics by 11.5% median across the 96 features, confirming it holds genuine TCGA pre-training statistics rather than being recomputed from an evaluation set.

Verdict: the published artifacts are authentic and mutually consistent. The agreement of NuCLS's seven class counts with Figure A.6 to the exact cell, and of the node and edge averages to two decimals, leaves no realistic doubt that these are the files used to produce the paper.

5 Results

5.1 Cell-level tasks — exact paper targets available

These are the settings where the paper tabulates specific values, so the comparison is unambiguous. Values are percentages, mean and standard deviation across folds.

Setting	Metric	Reproduced	Paper	Delta
PanNuke PanCancer	Macro F1	59.54 ± 0.08	59.47 ± 0.11	0.07
	Balanced acc	68.70 ± 0.98	68.73 ± 0.96	0.03
	AUROC	89.68 ± 0.04	89.57 ± 0.05	0.11
	AUPRC	66.45 ± 0.85	66.30 ± 0.94	0.15
PanNuke Breast	Macro F1	56.39 ± 0.06	56.43 ± 0.04	0.04
	Balanced acc	57.49 ± 0.07	57.49 ± 0.07	0.00
	AUROC	85.49 ± 8.13	85.30 ± 8.07	0.19
	AUPRC	61.94 ± 0.27	61.85 ± 0.30	0.09
NuCLS main	Macro F1	25.96 ± 2.51	26.57 ± 2.16	0.61
	Balanced acc	28.59 ± 2.85	28.06 ± 2.45	0.53
	AUROC	72.41 ± 1.68	71.49 ± 1.55	0.92
	AUPRC	29.51 ± 3.02	29.18 ± 1.61	0.33
NuCLS super	Macro F1	46.52 ± 2.53	46.24 ± 3.89	0.28
	Balanced acc	50.96 ± 5.28	48.04 ± 3.14	2.92
	AUROC	78.30 ± 2.58	77.14 ± 2.08	1.16
	AUPRC	52.93 ± 7.20	49.88 ± 2.59	3.05

PanNuke is reproduced. Eight metrics, maximum deviation 0.19 points, one exact match. The standard deviations track as well — including the conspicuous ± 8.07 AUROC spread on the breast subset, which reproduces as ± 8.13 . That figure looks like a typo in the paper until you run it: it is driven by a single low-performing third fold (73.99 against 91.56 and 90.91). Reproducing an anomaly of that shape is much stronger evidence than reproducing a well-behaved mean.

NuCLS is close but not exact, and the reason is known and structural — see Section 6.

5.2 Region-level tasks — Figure 4b

The paper’s Figure 4b is a bar chart with no tabulated values. We attempted to recover the bar heights from the PDF, but the figures are embedded as rasters whose extracted payload is a 404-byte placeholder, and the bars are absent from the vector layer. **No per-bar targets exist to compare against**, so we report our measurements against the patched Table 2 values that the paper claims performance should remain stable relative to.

Dataset	Macro F1	Balanced acc	AUROC	AUPRC	Table 2 (patched)	Delta
BreakHis	90.43	91.39	96.67	95.53	89.37	+1.06
Non-Patch BRACS	62.26	63.35	90.35	65.59	60.30	+1.96
Non-Patch						

The paper states that “the performance for BRACS and BreakHis remains remarkably stable across all patch sizes, including those on the non-patched image”, and concludes that one can “construct a single graph for an entire region of interest, without sacrificing performance”. Our measurements support this: both datasets

land within two points of the patched MIL pipeline, and in fact slightly above it, while bypassing patching and MIL aggregation entirely.

5.3 Structural claims verified

Claim	Source	Result
9.50M / 21.12M / 37.34M parameters at $d = 512 / 768 / 1024$	Table C.7	Exact, all three
96 node features	Table B.6	Confirmed (71 scalars + 25 Fourier coefficients)
PanNuke patch / node / edge statistics	Table 1	Matches to 2 decimals
NuCLS class distribution	Figure A.6	All 7 exact
BRACS and BreakHis class distributions	Figure A.5	All 9 exact

A note on the parameter count, because it is easy to get wrong. The checkpoint’s `state_dict` contains 17.68M values, which appears to contradict Table C.7. It does not: the ACM blocks are registered twice — once in `nns_lowpass` and again inside `ACM_convs[i]` — and `state_dict()` does not deduplicate shared modules while `parameters()` does. The 114 MB file size settles it independently: $9.5\text{M parameters} \times 4 \text{ bytes} \times (\text{weights} + \text{two Adam moments}) = 114 \text{ MB}$.

5.4 Coverage

Result family	Cells	Reproduced
Cell-level (Table 5, D.13–D.15)	24	16
Region-level non-patched (Figure 4b)	11	7
Efficiency (Table 4)	4	1
Region and slide MIL (Tables D.9–D.12)	48	0
Survival (Table 3, Figure 3)	2	0
Total	89	24 (27%)

6 Measured runtimes

All timings on an 11-core Apple M3 Pro laptop, CPU only.

Stage	Scale	Wall clock
PanNuke embeddings, 3 folds	7,208 graphs, 163,163 cells	20 s
PanNuke PanCancer probe	3 folds, ~110k train cells	130 s
PanNuke Breast probe	3 folds, tissue-filtered	24 s
NuCLS embeddings	1,694 graphs, 52,110 cells	7 s
NuCLS main probe	5 grouped folds, 7 classes	44 s
NuCLS super probe	5 grouped folds, 4 classes	3 s
BreakHis embeddings	522 RoI graphs	5 s
BreakHis probe	417 train / 105 test	4 s
BRACS embeddings	4,493 RoI graphs, ~8.5M nodes	10 m 34 s
BRACS probe	3,594 train / 899 test	7 s
Total compute		~13 minutes

Downloads dominated: roughly 4 GB of graph data, of which BRACS alone is 3.8 GB.

This is worth stating plainly because the paper’s most-quoted figure is 50 GPU-hours of pre-training. That number is real, but it describes producing the checkpoint. *Validating* the published downstream results from that checkpoint costs about thirteen minutes on a laptop. The two should not be confused.

Two operational constraints worth recording. BRACS requires `--batch_size 8`: at 1,888 nodes per graph on average and 15,496 at maximum, the default 2048 would need tens of gigabytes for the concatenated hidden states. And `graph_path` in the shipped `metadata.csv` files is relative to the repository root, so the scripts must be invoked from there rather than from `src/train`.

7 What we did differently, and why

This section is the honest accounting of every deviation. Each entry states what the paper did, what we did, why, and what effect it plausibly had.

7.1 Reconstructed NuCLS folds — the one deviation that moved numbers

Paper: uses the provided train/test splits, which for NuCLS are hospital-based.

Us: GroupKFold(`n_splits=5`) grouped on the TCGA tissue source site code.

Why: the NuCLS dataset card documents a `splits/` directory containing `fold_N_train.csv` and `fold_N_test.csv`. That directory is not in the repository — the URL returns HTTP 404, and the Hugging Face file listing contains only `.gitattributes`, `README.md`, `animation.gif` and `data/`. The paper’s exact fold assignment is therefore unrecoverable from published material.

Our substitute is principled rather than arbitrary. NuCLS folds are defined by hospital, and for TCGA slides the two-character code following TCGA- is the tissue source site, i.e. the contributing hospital. Grouping on it reproduces the *kind* of split the authors used — no slide from a hospital appears in both train and test — without reproducing the exact partition.

Effect: this fully explains the NuCLS deltas. Macro F1 lands within 0.61 (main) and 0.28 (super), which is well inside the paper’s own fold-to-fold standard deviations of ± 2.16 and ± 3.89 . The larger deltas on balanced accuracy (2.92) and AUPRC (3.05) for the super task are exactly the metrics most sensitive to how the rare classes distribute across folds, and with 124 slides across a handful of source sites, a different partition moves them by several points. **NuCLS should be read as consistent with the paper, not as an exact reproduction.**

7.2 Derived the NuCLS super labels

Paper: reports both the 7-class “main” and 4-class “super” NuCLS tasks.

Us: the released dataset contains main labels only, so we derived super by the documented NuCLS aggregation: tumour mitotic and non-mitotic merge to Tumour Any; lymphocyte and plasma cell merge to sTIL; stromal and macrophage merge to Non-TIL stromal; other passes through.

Confidence: certain. The derived distribution reproduces all four super-class counts in Figure A.6 exactly — 23,369 / 17,261 / 10,893 / 587. A wrong mapping could not produce four exact matches.

7.3 Non-patched rather than patched region graphs

Paper (Table 2): BACH, BRACS and BreakHis patched into 224px tiles, embedded per tile, aggregated with MIL.

Us: whole-RoI graphs, one per image, non-linear probing.

Why: the patched graphs were never released, only the whole-RoI ones. Rebuilding the patched versions needs the original BACH, BRACS and BreakHis images plus a full re-run of segmentation, feature extraction and graph construction.

Effect: our region-level numbers answer Figure 4b’s question, not Table 2’s. We have labelled them accordingly throughout rather than presenting them as Table 2 reproductions.

7.4 Wrote the PanNuke tissue filter

Paper: reports PanNuke on all pan-cancer tissues and on the breast subset separately.

Us: the repository has no mechanism to select a tissue subset, so we added `--tissue_filter`. Filenames carry the tissue as a prefix, making this a one-line selection.

Effect: none on correctness. The breast column reproduces to within 0.19 points, which confirms the filter selects the intended subset.

7.5 Ran on CPU with a much newer software stack

Effect: none detectable. See Section 3.2 for the rationale. The PanNuke agreement to 0.19 points across a five-release PyG jump and a different accelerator is itself evidence that the published results are not environment-dependent.

7.6 Did not pre-train, and did not build the baselines

Why: out of scope by design. Pre-training would test a different claim (that the recipe reproduces), and it needs the TCGA corpus. The baselines have no training code in the repository, so building them would be re-implementation rather than reproduction.

Consequence: this report validates GrapHist’s published downstream numbers. It says nothing about whether GrapHist *beats* DINOv2 or MAE, because we did not run those models. The comparative claims of the paper remain untested here.

8 Defects found in the released code

A fresh checkout cannot produce the results in Section 5. Four defects fire in sequence. All were fixed in our working copy; the original is preserved at `src.orig.bak`.

8.1 Cell labels are misaligned against their embeddings — silent

Severity: critical. `generate_embeddings_cell` slices `batch.labels` using `batch.ptr`. But `ptr` indexes the node dimension, which `AddVirtualNode` has padded with one extra node per graph, whereas `labels` was never padded. Graph i within a batch is preceded by exactly i virtual nodes, so its label slice is displaced by i positions.

i	<code>n_real</code>	<code>ptr[i]</code>	<code>code slice</code>	<code>correct slice</code>	<code>labels correct?</code>
0	31	0	[0:31]	[0:31]	yes
1	19	32	[32:51]	[31:50]	NO
2	22	52	[52:74]	[50:72]	NO
7	15	170	[170:185]	[163:178]	NO

Only the first graph of each batch is correct. The shape check downstream in `main_cell.py` catches this **only** for graphs near the end of a batch, where the slice overruns the label array; for everything earlier the length is right and the labels are simply wrong. The paper’s cell-level numbers can only have been produced with `batch_size=1`, where i is always zero. The shipped default is 2048.

Fix: offset the label slice by `start - i`. Verified against ground truth for all 7,208 PanNuke and 1,694 NuCLS graphs.

8.2 AUROC assumes every class appears in every fold — silent then fatal

Severity: high. `train_and_eval_cell_level` calls `roc_auc_score(..., multi_class="ovr")`, which requires every class to be present both in the fitted classifier and in the evaluation set. Grouped folds violate this for rare classes — NuCLS “Other” has 587 cells across 124 slides and vanishes from entire folds — producing `ValueError: Number of classes in y_true not equal to the number of columns in y_score`.

Fix: average per-class one-vs-rest scores over the classes that are well defined in each fold. This is mathematically identical to sklearn’s macro average when nothing is missing, which we confirmed by re-running PanNuke as a regression test: identical to 0.01.

8.3 The checkpoint cannot be loaded without a GPU

`load_checkpoint` calls `torch.load` with no `map_location`. Since `graphist.pt` was saved from CUDA, any CPU-only or Apple Silicon machine fails immediately with “*Attempting to deserialize object on a CUDA device*”.

8.4 The normalization transform cannot be pickled

`NormalizeData.__init__` builds `defaultdict(lambda: defaultdict())`. A local `lambda` is unpicklable, so any `DataLoader` spawning workers fails. The default is `--num_workers 4`, so this fires immediately on macOS and Windows, which use `spawn`; Linux escapes only because it defaults to `fork`.

8.5 Version incompatibilities

Symptom	Cause
<code>propagate()</code> got an unexpected keyword argument 'edge_weight'	The <code>propagate_type</code> annotation declares <code>edge_attr</code> while the call and <code>message()</code> use <code>edge_weight</code> . Harmless on PyG 2.5.2, fatal on 2.8.

Symptom	Cause
LogisticRegression() got an unexpected keyword argument 'multi_class'	Deprecated in scikit-learn 1.5, removed in 1.7. Multinomial is the lbfgs default, so removal is behaviour-preserving.

8.6 Placeholder paths

`main_patch.py` hardcoded all three of its paths to `PATH/T0/...` strings and used `--dataset` only as a printed label, so the invocation documented in the README could not work. `pretrain.py` still contains `run_name = "YOUR/RUN/NAME"`, whose embedded slashes make the first checkpoint save resolve into a non-existent directory — it would crash after a full epoch of training.

8.7 Summary of changes made

File	Added	Removed	Purpose
<code>evaluate/main_cell.py</code>	188	30	Tissue filter, group folds, super labels, robust metrics
<code>evaluate/utils.py</code>	51	0	Shared metric and device helpers
<code>evaluate/main_slide.py</code>	47	30	Balanced accuracy, AUROC, AUPRC; device fallback
<code>evaluate/main_patch.py</code>	30	14	CLI paths replacing placeholders; three metrics
<code>train/generate_embs.py</code>	13	2	Label alignment; MPS/CPU fallback
<code>train/utils.py</code>	10	4	<code>map_location</code> ; picklable transform
<code>train/models/acm_gin.py</code>	1	1	<code>propagate_type</code> annotation

The three missing metrics deserve separate mention: the paper reports balanced accuracy, AUROC and AUPRC for slide- and region-level tasks in Tables D.10, D.11 and D.12, but `main_slide.py` computed only macro F1 and loss. Those tables were unreachable from the released code regardless of data availability.

9 A substantive discrepancy: the virtual node

This is separate from the defects above because it is not a bug in the ordinary sense — it is a divergence between what the paper describes and what the code does, and it is baked into the released weights.

What the paper says. The virtual node is “connected to all nodes in each graph. The virtual node’s features are zero-initialised, and its edge weights are set to the mean of all edge weights in the pre-training dataset.”

What the code does. `AddVirtualNode` hardcodes an edge weight of 22.5. Crucially, the transform pipeline is ordered `ToUndirected` → `NormalizedData` → `AddVirtualNode`, so that raw-micron constant is injected **after** the edge weights have been z-scored.

The magnitude. The downloaded `normalization.json` pins the pre-training mean edge weight at 18.627 μm with a standard deviation of 12.735. The correct normalized value is therefore $(22.5 - 18.627)/12.735 = 0.304$. The code uses 22.5 — roughly **74 times too large**. Measured on real batches, genuine normalized edge weights span -1.29 to $+5.10$ against that constant. Back-transformed, 22.5 corresponds to about 305 μm , which is three times the 100 μm biological cutoff the graph construction enforces.

Why it matters. The ACM block normalizes by *weighted* degree. With one edge of weight 22.5 and a handful of real edges averaging 0.26, over 90% of a typical node’s aggregation mass comes from the virtual node. The low-pass and high-pass channels are therefore reading something close to a graph-wide mean rather than a local cellular neighbourhood — which is the mechanism the paper’s entire argument rests on.

What this does and does not imply. It does *not* invalidate the results: they reproduce, and the model demonstrably works. What it means is that the published model may be succeeding for a different reason than the paper’s stated mechanism, and that a corrected virtual node is an untested and potentially significant improvement. This is, in our assessment, the single most interesting open question in the codebase, and Section 5 now provides a precise baseline against which to measure it.

There is a related latent hazard. The weighted-degree division is guarded by `masked_fill_(deg_inv == inf, 0)`, which catches an exactly-zero degree but not a near-zero one — and z-scored edge weights are signed, so a degree can cancel toward zero. The measured minimum degree was 17.6, safe only *because* the 22.5 constant dominates the sum. Correcting the virtual node removes that accidental protection, so the guard should be changed to a magnitude clamp at the same time.

10 Other observations

Seven declared hyperparameters never reach the model. `residual`, `norm`, `negative_slope`, `attn_drop`, `in_drop`, `dropout` and `num_heads` are parsed, threaded through `build_model`, accepted by `PreModel.__init__` — and then dropped. `ACM_GIN_model` references none of them. `create_norm` and `NormLayer` are dead code. The practical consequence is that **GrapHist pre-trains with no dropout and no normalization layers of any kind**, since `--batchnorm` also defaults to `False`. Setting `--in_drop 0.2` as the README suggests changes nothing.

“Non-linear probing” is a linear probe. The paper uses the phrase three times and then specifies logistic regression; the code confirms plain `LogisticRegression` inside a `StandardScaler` pipeline. Terminology only — the code and the stated method agree with each other.

MIL embedding normalization is computed, saved, then discarded. `EmbeddingsDataset._compute_normalizer_values` makes a full streaming pass, writes `normalization.json`, stores the statistics — and `__getitem__` returns the raw tensor. The MIL heads consume unnormalized embeddings.

Slide and patch embeddings pool the virtual node in. The paper defines a patch embedding as the mean of its *cell* embeddings, but `generate_embeddings_slide` and `generate_embeddings_patch` mean over all nodes including the virtual one. `generate_embeddings_cell` is the only path that correctly excludes it.

A quarter of the MIL grid is redundant. Dropout is inserted only when the classifier has a hidden layer, and the attention module has none, so with `clf_hidden_dim=[]` the two dropout values are identical runs — 4 of 16 configurations duplicate, per dataset, per aggregator.

The released PanNuke graphs are scaled as 20x, not the 40x the card states. Median nuclear area is 26.4 μm^2 and median inter-nuclear distance 13.3 μm , both physically sensible only at 0.5 $\mu\text{m}/\text{pixel}$ — the value hardcoded in the feature extraction script. At a true 40x of 0.25 $\mu\text{m}/\text{pixel}$ the implied nucleus would be 2.9 μm across, which is not biological. Because features are z-scored downstream this does not affect training, but it does affect the 100 μm edge cutoff and any cross-dataset comparison.

Edges are stored one-directional. Every `edge_index` has `src < dst`. The training pipeline applies `ToUndirected()`, so it is fine there, but the dataset card’s quick-start example loads graphs raw. The quoted average edge counts are single-direction; message passing sees double.

11 Conclusions

The published downstream results reproduce. PanNuke’s eight metrics land within 0.19 points, including a reproduced fold-level anomaly. NuCLS lands within about one point on macro F1 under a fold assignment we had to reconstruct because the documented split files were never published. The region-level datasets confirm the paper’s patch-size robustness claim. Across a very different software stack and on CPU only.

The released artifacts are genuine. Node and edge statistics match Table 1 to two decimals; class distributions match Figures A.5 and A.6 exactly across three datasets; parameter counts match Table C.7 exactly at all three widths; the normalization statistics are demonstrably from the pre-training corpus rather than recomputed.

The released code does not run as shipped. Four defects block a fresh checkout, and the most serious is silent: anyone running the shipped defaults would obtain corrupted cell-level labels without an error. Three of the paper’s own reported metrics were never implemented for slide- and region-level tasks, making Tables D.10–D.12 unreachable from the repository as published.

One methodological discrepancy warrants follow-up. The virtual-node edge weight is applied in the wrong space, by a factor of roughly 74, with the consequence that local neighbourhood aggregation is largely swamped by a global term. The results stand, but the stated mechanism may not be the operative one.

Coverage is 24 of 89 GrapHist result cells. The ceiling is set by artifact availability rather than by compute: BACH is unpublished, and the region-level graphs that were published are the non-patched variants. Only the TCGA-BRCA path — now known to be published — offers a substantial extension without needing original images.

11.1 Recommended next steps

1. **Inspect the TCGA-BRCA archive.** It is published, contrary to our earlier assumption. If it contains usable graphs, the WSI column and survival analysis become reachable without rebuilding from slides.
2. **Run the virtual-node ablation.** Change 22.5 to 0.304, add the magnitude clamp to the degree division, and re-run the PanNuke probe against the exact baseline in Section 5. One line, and the most informative experiment available.
3. **Report the label-alignment defect upstream.** It silently corrupts cell-level results for every user running the shipped defaults.
4. **Request the NuCLS splits/ directory** from the authors. It is documented in the dataset card but absent from the repository, and it is the only thing standing between our NuCLS numbers and an exact reproduction.
5. **Treat the remaining tables as re-implementation, not reproduction.** BACH and the four baseline models require writing code the repository does not contain.

12 Appendix: reproduction commands

Run from the repository root, since graph_path in the shipped metadata.csv files is relative to it.

```
# --- PanNuke: embeddings for each fold ---
for F in 1 2 3; do
    python3 src/train/generate_embs.py --dataset PanNuke --scale cell \
        --num_workers 0 \
        --model_path weights/graphist.pt \
        --norm_json_path normalization.json \
        --cell_graphs_path graph-pannuke/data/fold_${F} \
        --output_path out/emb_pannuke/fold_${F}
done

# --- PanNuke: pan-cancer and breast-only probes ---
python3 src/evaluate/main_cell.py --split_mode folder \
    --fold_dirs out/emb_pannuke/fold_1 out/emb_pannuke/fold_2 out/emb_pannuke/fold_3 \
    --save_dir out/res_pannuke_pancancer

python3 src/evaluate/main_cell.py --split_mode folder \
    --fold_dirs out/emb_pannuke/fold_1 out/emb_pannuke/fold_2 out/emb_pannuke/fold_3 \
    --tissue_filter Breast --save_dir out/res_pannuke_breast

# --- NuCLS: embeddings, then main and super tasks ---
python3 src/train/generate_embs.py --dataset NuCLS --scale cell --num_workers 0 \
    --model_path weights/graphist.pt --norm_json_path normalization.json \
    --cell_graphs_path graph-nucls/data --output_path out/emb_nucls

python3 src/evaluate/main_cell.py --split_mode group --emb_dir out/emb_nucls \
    --group_regex '^(TCGA-[A-Z0-9]+)' --num_folds 5 --save_dir out/res_nucls_main

python3 src/evaluate/main_cell.py --split_mode group --emb_dir out/emb_nucls \
    --group_regex '^(TCGA-[A-Z0-9]+)' --num_folds 5 \
    --label_map nucls_super --save_dir out/res_nucls_super

# --- Region level, non-patched (BRACS needs the small batch size) ---
python3 src/train/generate_embs.py --dataset BreakHis --scale patch \
    --num_workers 0 --batch_size 64 \
    --model_path weights/graphist.pt --norm_json_path normalization.json \
    --metadata_csv_path graph-breakhis/metadata.csv --output_path out/emb_breakhis

python3 src/train/generate_embs.py --dataset BRACS --scale patch \
    --num_workers 0 --batch_size 8 \
    --model_path weights/graphist.pt --norm_json_path normalization.json \
    --metadata_csv_path graph-bracs/metadata.csv --output_path out/emb_bracs

python3 src/evaluate/main_patch.py --dataset BreakHis --seed 0 \
    --metadata_csv graph-breakhis/metadata.csv --embs_path out/emb_breakhis \
    --save_dir out/res_breakhis_nonpatch

python3 src/evaluate/main_patch.py --dataset BRACS --seed 0 \
    --metadata_csv graph-bracs/metadata.csv --embs_path out/emb_bracs \
    --save_dir out/res_bracs_nonpatch
```

Datasets were obtained by git clone from <https://huggingface.co/datasets/ogutsevda/pannuke> for graph-pannuke, graph-nucls, graph-bracs and graph-breakhis.